

System Architecture Overview

Purpose

Our program allows users to play the game “Minesweeper” on a 10x10 board in the terminal. The user can choose between 10 and 20 mines to be randomly placed on the board for each game. The game supports left and right clicks through a mouse or can be played completely using only keyboard input. The program also gives the user the option to play again at the conclusion of a game. There are 3 separate program builds, one for Linux, another for Mac, and a third for Windows.

Components

This implementation of Minesweeper is composed of several python modules. At the top level, we have two: **src** and **test**. The test module is built using pytest and is intended to provide an automated sequence of unit tests that can be executed to validate that our program functions as expected. The src module contains two main python files, `main.py` and `classes.py`. The src module also has a submodule called **tui** which contains the `run_tui.py` file.

The program is primarily contained within two python files, “`classes.py`” and “`run_tui.py`”. The “`classes.py`” file acts as the backend of the project, containing the necessary functions and classes to organize and handle game logic related tasks. The “`run_tui.py`” file then acts as the frontend displaying the game information and state to the user in the terminal and processing user input.

We will explain in more detail later in this document, but the main logical organization of our program’s components is split up between the frontend and backend. The frontend is responsible for displaying the user interface, processing user input, passing information to the backend, receiving updates, and then displaying those changes back to the user. The backend is responsible for taking those updates, processing them according to the rules of Minesweeper, and then responding with those results.

These responsibilities can be summarized as follows: the frontend handles the user interface and input handler, while the backend handles the board manager and game logic. The frontend is mostly contained with the `run_tui.py` file and its Frontend class, while the backend is primarily stored in the `classes.py` file and its GameManager class.

Outside of the game-specific components and the related Python modules we've created, we also used a tool called *pyinstaller* to create executable versions of our project. These executables are provided in the *dist-<operating system>* folders. The scripts used to generate these files are provided in the main project directory called *make-<operating system>.sh/.bat*. Choose the appropriate script, run it, and new executables will be generated. It will also generate the *build-<operating system>* and *spec-<operating system>* folders. These contain configuration and intermediate build output files that may be useful for debugging purposes.

The project's dependencies are listed below:

- Python 3
- curses (Linux) / windows-curses (Windows)
- Standard python libraries: enum, platform, random
- pyinstaller (to build the project)
- pytest (to run tests using the test module)

Curses is a Python library that allows for the development of text-based user interfaces in the terminal. Instead of printing lines of texts that scroll down, curses offers control of the entire terminal window. It allows cursor movement to any position, color changes, keyboard and mouse input, and updates to parts of the screen without clearing everything. This is what allows the game to have a proper grid interface where the user can click on cells and see updates in place.

Data Flow

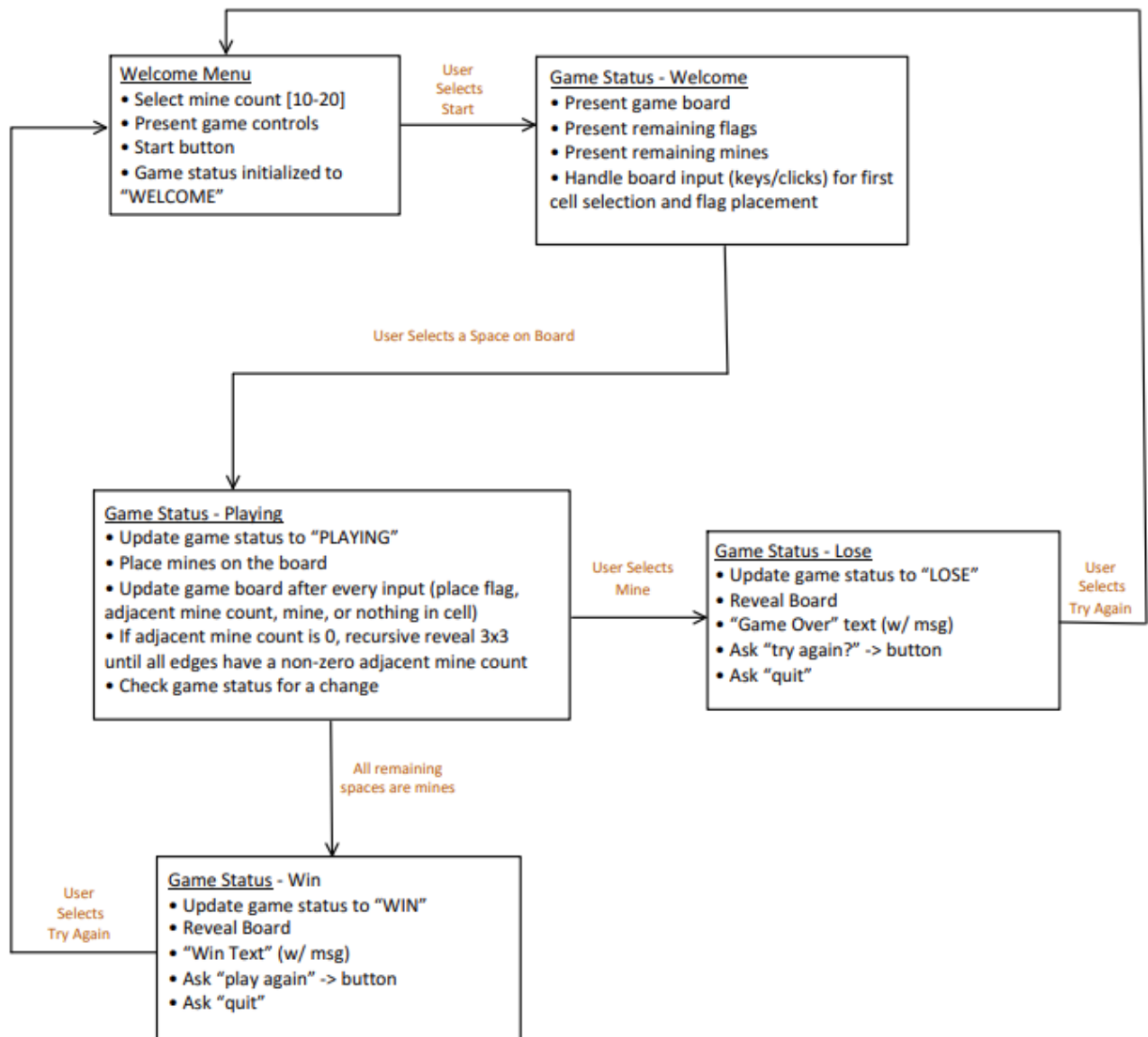


Figure 1

This diagram presents a flowchart of all possible game states within our Minesweeper implementation. It illustrates the program's progression based on player input, beginning at the welcome menu, transitioning into active gameplay upon the first cell selection, and concluding with either a win or loss screen. The flowchart highlights core processes such as mine placement, board updates after each action, and the system's response to different outcomes.

Key Data Structures

The program uses multiple classes to represent the different parts of the game. There are 5 total classes that are used in our implementation. They are **GameManager**, **Frontend**, **Cell**, **CellState**, and **GameStatus**.

GameManager: GameManager can be thought of as the game's backend. It handles gameplay logic and stores key game-related information, such as the gameplay board represented as a 2D array of rows and columns. This array stores a Cell object for each space in that 2D array (the "grid").

Frontend: Frontend is the class responsible for processing user input and passing the necessary information back to the GameManager. It also retrieves updates from GameManager and updates the user interface to show gameplay updates (e.g. changes to the game board) to the player.

Cell: A **Cell** is used to represent a singular square on the board, and it contains state information about itself, such as the number of squares with adjacent mines or if the Cell itself is flagged. As mentioned previously, the game board is contained within GameManager and is a 2D array of Cells.

CellState: Each Cell also has its own CellState object, which is an enumeration that tracks whether the Cell has no adjacent squares with mines, at least one adjacent square with a mine, or if it has a mine on it.

GameStatus: The GameManager also has a GameStatus object, which is an enumeration that keeps track of the game's status so that the Frontend can handle the visuals accordingly. For example, if the game is still ongoing the GameStatus would be PLAYING. If the player then makes a move which loses the game, the GameStatus would transition to LOSE, and the Frontend would display all the Cells on the board.

The diagram below (Figure 2) outlines each class in the program and its usage.

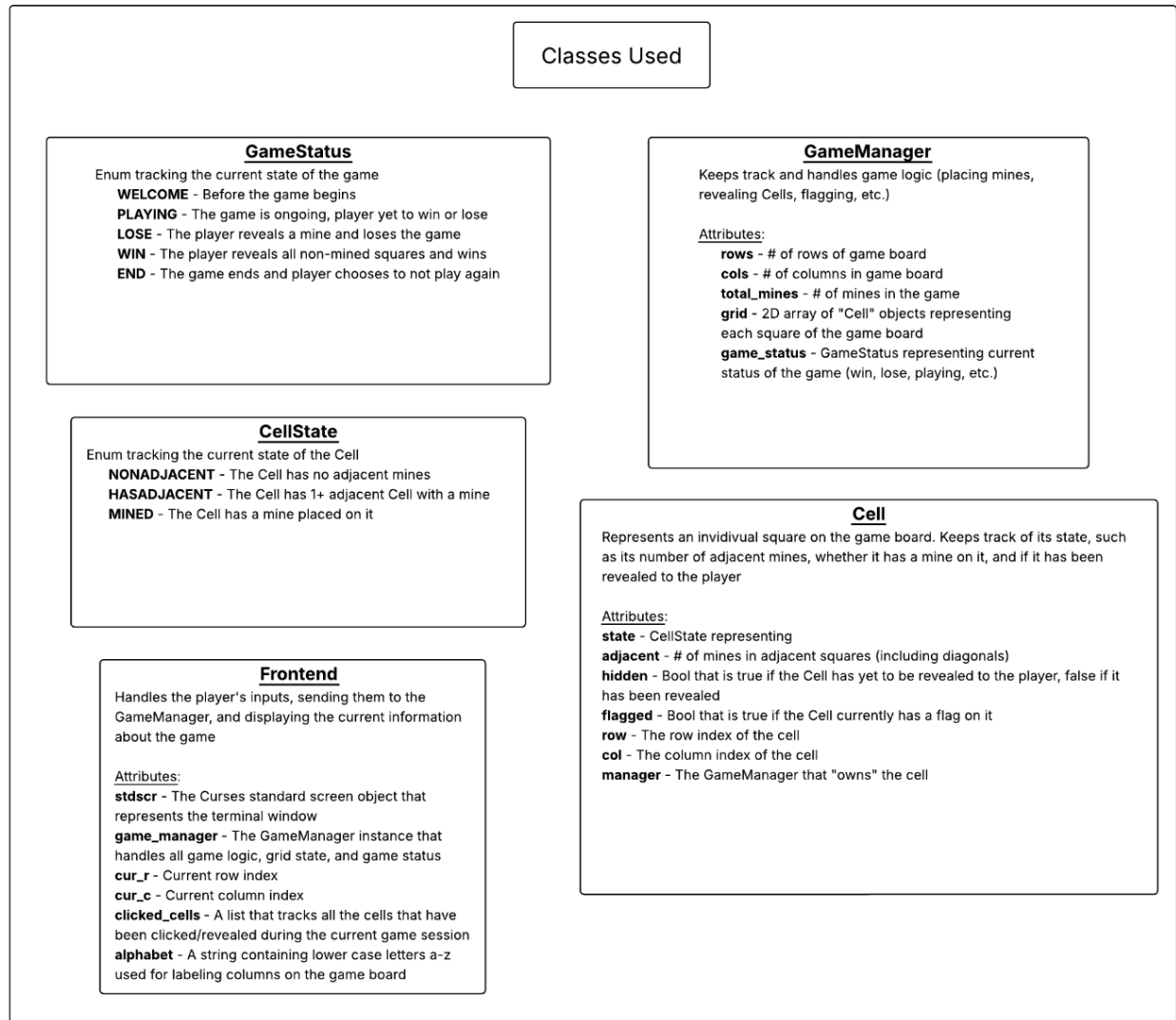


Figure 2

The diagram below (Figure 3) shows the class inheritance hierarchy.

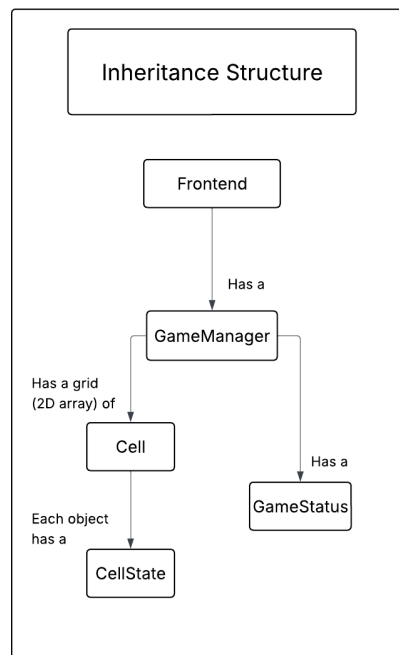


Figure 3

Assumptions

We make the following design assumptions:

Gameplay:

- A fixed 10x10 size grid for gameplay.
- The user will specify a number of mines between 10-20 at the start of the game.

User technical skills:

- The user will know how to download and run the executable from the GitHub repository and build from source if necessary.
- The user will have a valid terminal interface that is properly configured.
- The user will be able to download any dependencies they need.

EECS 581 — Project 2: Minesweeper Maintenance

System Architecture Addendum

Team 26

October 5, 2025

Overview

This addendum documents **four concrete changes** to the inherited terminal Minesweeper system and references the exact classes, methods, and logic we added or modified:

1. **Timer** (start/stop semantics; UI rendering; persistence strategy)
2. **High Score** (per-mine-count table; update criteria; UI message)
3. **AI Modes** (Easy/Medium/Hard; automatic mode; turn alternation)
4. **Audio Effects** (event→WAV mapping; non-blocking playback)

Core gameplay behavior (first-click safety, flood reveal, win/loss rules) remains unchanged.

Timer

Location & Fields

Backend timing state lives in `src/classes.py` within class `GameManager`:

- `self.start_time`: float (seconds since epoch; initially 0)
- `self.finished_time`: float (captured on win; initially 0)

Start/Stop Semantics

- **Start:** in `GameManager.handle_clicked_cell(i,j)`, on the *first effective reveal* (transition to `GameStatus.PLAYING`), we set `self.start_time = time.time()`.
- **Stop (Win):** when `check_win()` returns true, we set `self.finished_time = time.time()` prior to WIN.
- **Loss:** time is not used for scoring on loss; no finished time stored.

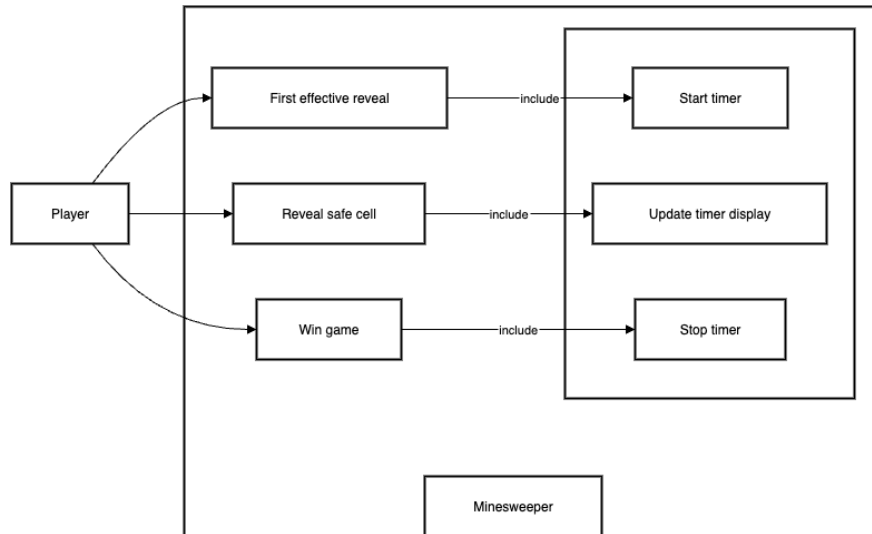


Figure 1: UML (use-case/state): Timer logic — start on first reveal, update while playing, stop on win.

UI Rendering

Frontend rendering lives in `tui/run_tui.py` within class `Frontend`:

- `draw_timer()`: if `GameStatus == WELCOME`, displays 0; otherwise displays `elapsed = [time.time() - game_manager.start_time]`.
- `draw_board()` calls `draw_timer()` each frame, so the counter updates continuously (paired with `stdscr.nodelay(True)` in `main.py`).

Edge Cases

- Timer starts only once (first effective reveal; guarded by `is_first_click`).
- Resets on `Frontend.reset_game()` (new `GameManager` instance).

High Score

Location & Fields

Per-mine-count table lives in `tui/run_tui.py`, class `Frontend`:

- `self.high_score`: `List[int]` length 11, initialized to a large sentinel (999999999) for mine-counts 10–20.
- **Indexing**: position $k = \text{total_mines} - 10$.

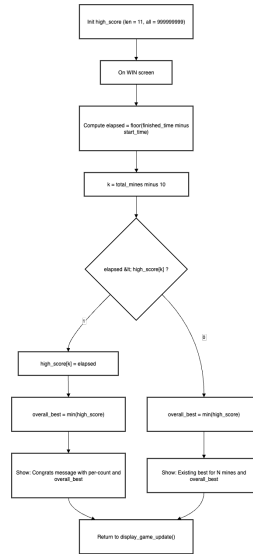


Figure 2: UML (activity): High-score logic — compute elapsed, compare per mine count, update and display banner.

Update Criteria (Win Screen)

In `Frontend.display_win_screen()`:

```
elapsed = [game_manager.finished_time - game_manager.start_time].
```

If $\text{elapsed} < \text{high_score}[k]$, update $\text{high_score}[k] \leftarrow \text{elapsed}$ and display: “*Congrats! New high score for N mines: elapsed. High score for any mine count: min(high_score).*” Else, display the existing per-count best and overall best.

Display Semantics

- Messages are assembled and passed to `display_game_update(msg_obj)`.
- The high score is shown only on **win**; on **loss**, no high-score update occurs.

Persistence Note

Current scope is in-memory for the demo; persistence (e.g., JSON file per OS) is a small extension point.

AI Modes

Location & Fields

Controller lives in `AI_mode.py`. Frontend integration points (in `tui/run_tui.py`):

- Construction: `self.ai_mode = AI_mode(self.game_manager)`
- Mode setters: `set_mode_easy()`, `set_mode_medium()`, `set_mode_hard()`
- Turn flags: `is_turn` (AI vs. player), `is_automatic_solver` (auto mode)
- Turn control: `ai_turn()` → returns the move cell; `change_turn()`

Start Screen & Keybindings

Handled in `Frontend.draw_start_screen()` and `start_game()`:

Key	Action
Enter/Return	Start player-only
1	Start multiplayer AI (Easy)
2	Start multiplayer AI (Medium)
3	Start multiplayer AI (Hard)
4	Automatic Easy (sets mode + calls <code>automatic_initializer()</code>)
5	Automatic Medium
6	Automatic Hard

Medium Mode Logic

The logic for the Medium mode was the most complex, and was accomplished by Aryan Kevat of Group 26. He accomplished this through iteratively reproducing a better version, utilizing a paper that documented minesweeper algorithms and optimization. The paper can be found here: <https://minesweepergame.com/math/a-simple-minesweeper-algorithm-2023.pdf>

Turn Alternation & Automatic Mode

- During `PLAYING`, if `ai_mode.mode != NONE` and `ai_mode.is_turn` is true, `start_game()` calls `ai_turn()`.
- After a valid player reveal (in `handle_left_click`), if the game is not over, we call `ai_mode.change_turn()` to hand the turn to the AI.
- `automatic_initializer()` sets `is_automatic_solver = True`, `is_turn = True`, then performs `ai_turn()` to kick off auto-solving.
- `temp_highlight()` provides a short visual cue on the AI's chosen cell.

AI Logic (Explicit Rules)

The AI *selects moves* while **all rule enforcement** remains in the backend (`GameManager.handle_clicked_cell`).

Easy Pick any hidden, unflagged cell uniformly at random.

Medium (Deterministic Rules) For each revealed cell v with number n , let $H(v)$ be hidden neighbors and $F(v)$ flagged neighbors:

1. **All Mines:** If $|H(v)| = n - |F(v)|$, *flag all cells in $H(v)$.*
2. **All Safe:** If $|F(v)| = n$, *open all cells in $H(v)$.*

If no rule applies, fall back to random.

Hard (Pattern Augmentation) Apply Medium rules, plus **1–2–1** side-by-side pattern detection:

- Deduce outer hidden neighbors are mines $\Rightarrow$ flag them.
- Deduce the inner hidden neighbor is safe $\Rightarrow$ open it.

Fallback remains Medium/random when patterns are absent.

Backend Safeguards

`GameManager.handle_clicked_cell(i,j)` validates all moves:

- **Flags/Hidden gating:** ignores flagged or already revealed cells.
- **First-click safety:** calls `generate_mines(i,j)` with exclusion; sets `PLAYING` and `start_time`.
- **Reveal semantics:** `reveal_cell` for numbered cells; `rec_reveal` flood for zero-adjacent.
- **Win:** on success, set `finished_time`, `WIN`, and `reveal_all`.
- **Loss:** reveal all and set `LOSE`.

Audio Effects

Location & Assets

Sound manager implemented in `src/classes.py` as class `SoundManager`:

- WAV files in `assets/sounds/`: `reveal.wav`, `flag.wav`, `win.wav`, `lose.wav`.
- Preload via `simpleaudio.WaveObject.from_wave_file(path)` into `self.waves`.

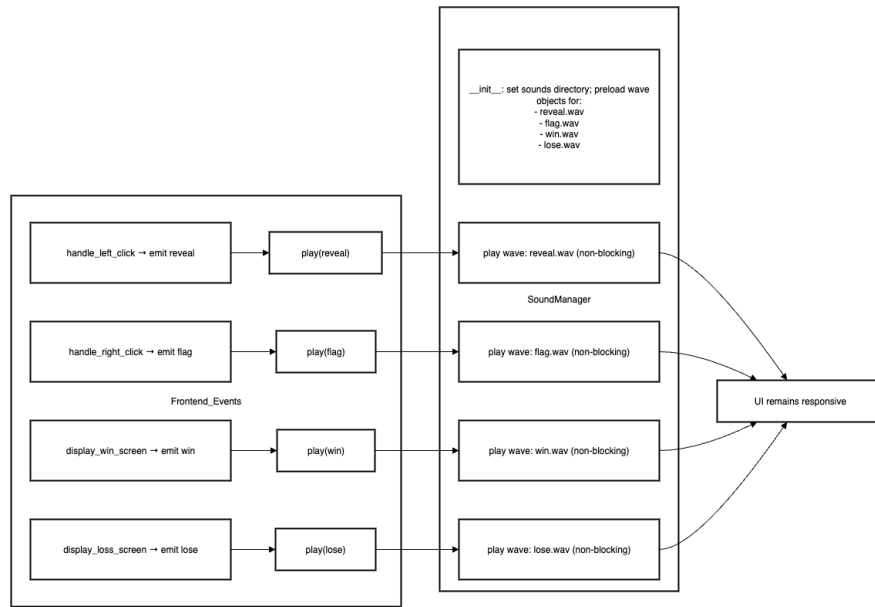


Figure 3: UML (use-case): Sound logic — event to SoundManager mapping (reveal/flag/win/lose).

Event Triggers

- `Frontend.handle_left_click`: `self.sound.play('reveal')` on a successful reveal.
- `Frontend.handle_right_click`: `self.sound.play('flag')` on toggle flag.
- `Frontend.display_win_screen`: `self.sound.play('win')`.
- `Frontend.display_loss_screen`: `self.sound.play('lose')`.

Non-Blocking Playback

`SoundManager.play(key)` obtains the `WaveObject` and calls `.play()` (non-blocking). UI responsiveness is preserved (no busy waits).

Audio Install/Help

Audio dependency: Install via `pip install simpleaudio`. Ensure WAV assets reside at `assets/sounds/`.

Implementation imports: `import os` and `import simpleaudio as sa`.

Compatibility & Packaging

- **Curses:** configured in `main.py` via `setup_curses(stdscr); stdscr.nodelay(True)` enables live timer updates.
- **PyInstaller:** ensure `assets/sounds/` are included by the OS-specific `make-<os>.sh/.bat`.

- **Windows:** use `windows-curses`; audio requires WAV support.

Testing Checklist (Targets & Methods)

Timer/High Score

- First effective reveal sets `start_time`; win sets `finished_time`.
- `elapsed = floor(finished - start)` computed as displayed.
- Index math: $k = \text{total_mines} - 10$ updates the correct slot.
- High-score message branches (new record vs. existing best).

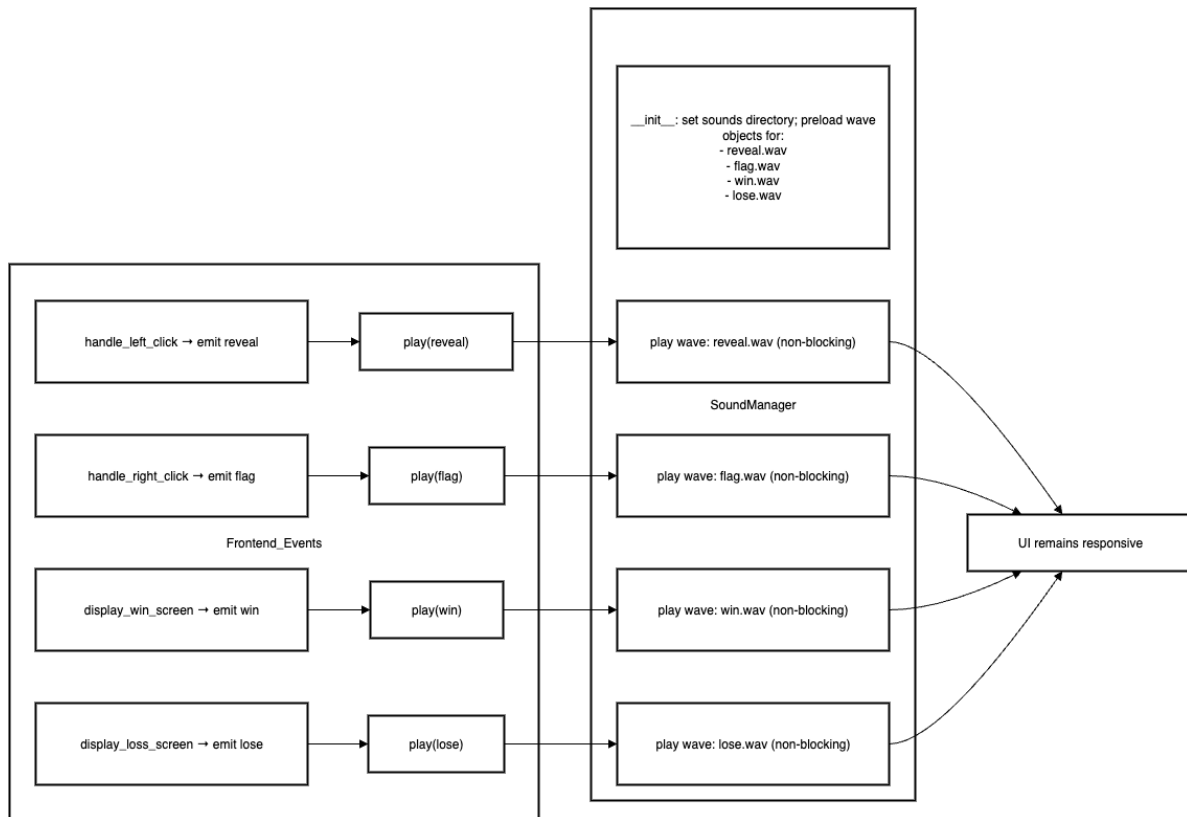
AI Modes

- Mode keybindings map correctly; auto modes invoke `automatic_initializer()`.
- `ai_turn()` respects `is_turn` and yields valid hidden/unflagged cells.
- Medium rules fire on crafted boards (all-mines/all-safe).
- Hard pattern (1–2–1) triggers when present; falls back when absent.

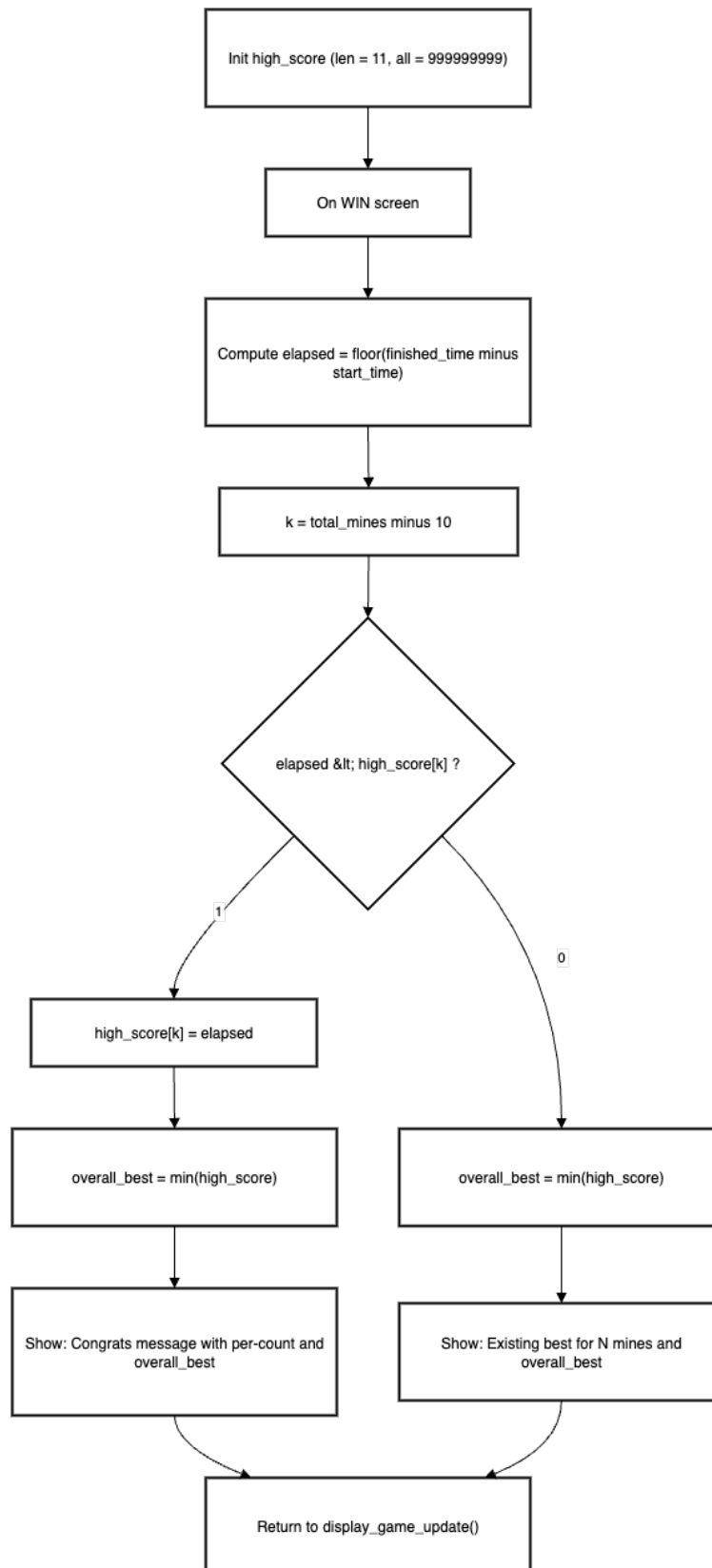
Audio

- All four events trigger the expected WAV; missing device/file does not crash.

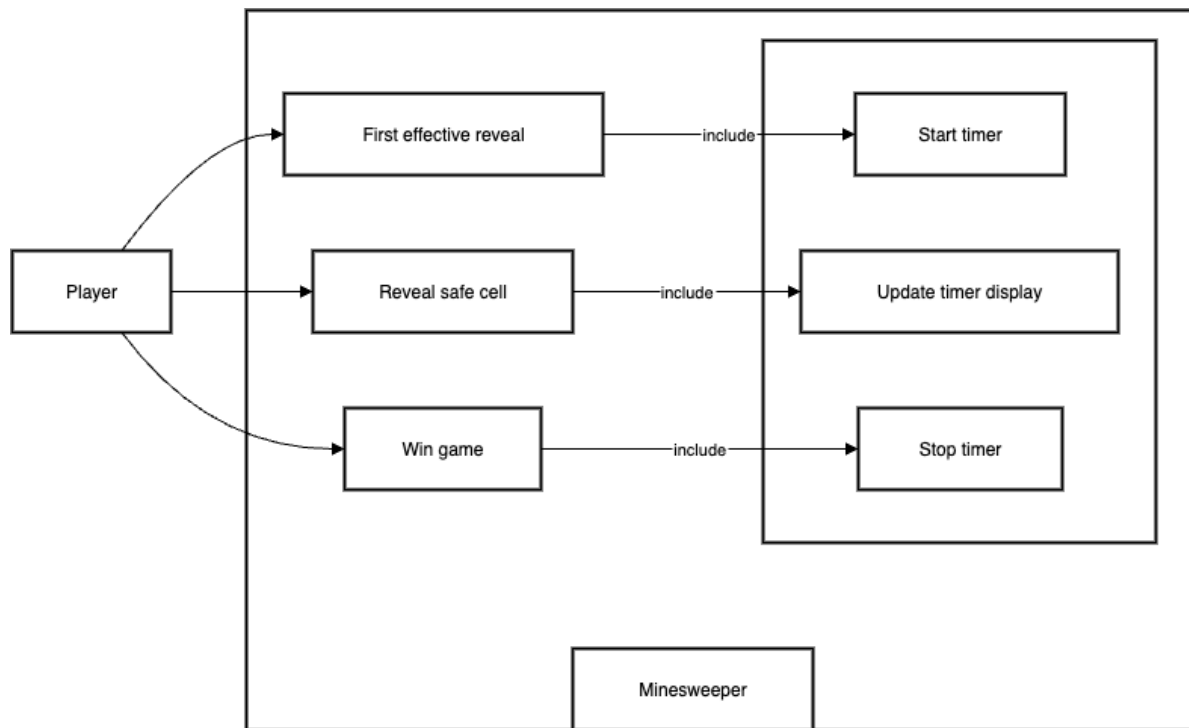
UML Diagrams (Reproduced)



(reproduced) See Figure 3: Sound logic.



(reproduced) See Figure 2: High-score logic.



(reproduced) See Figure 1: Timer logic.